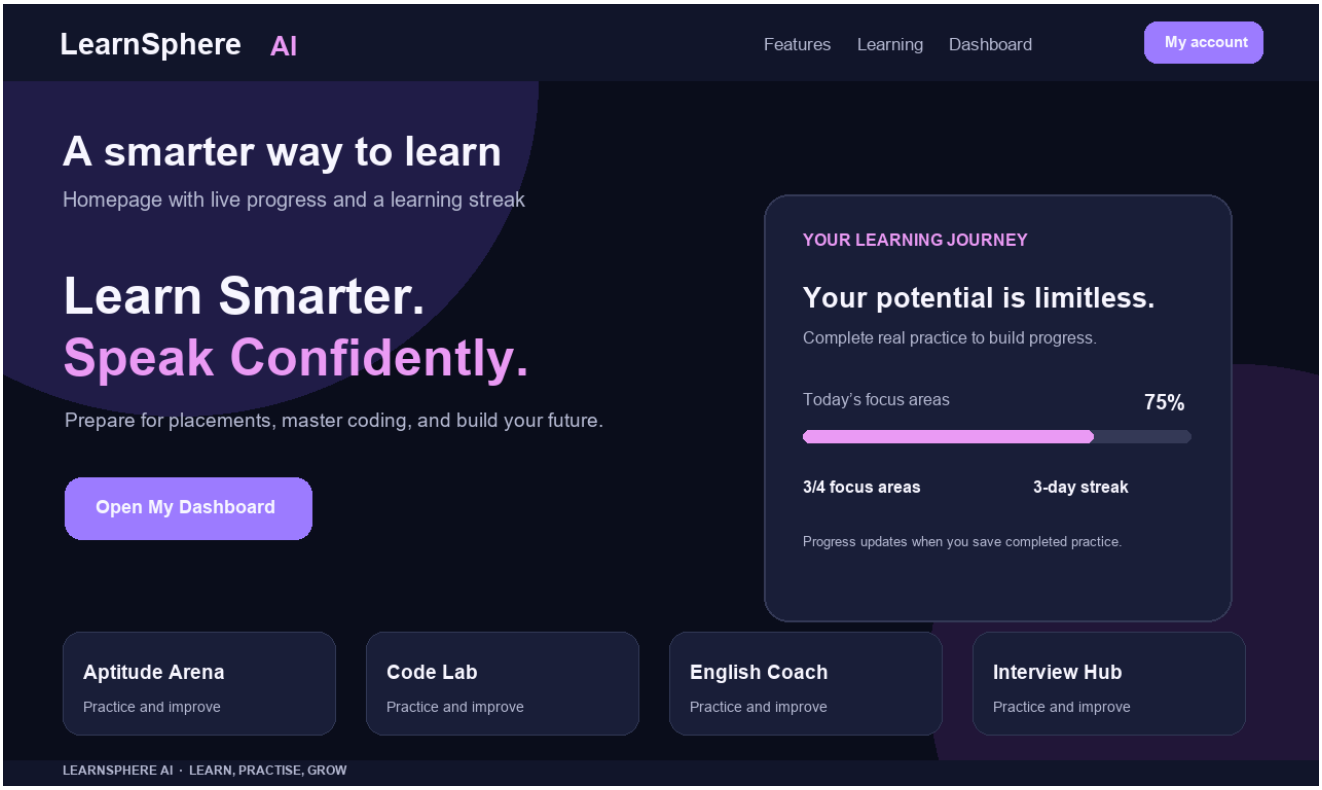
PROJECT EXPLANATION

LearnSphere AI

A clear guide to the learning platform, its website pages, technology, and deployment.



Homepage visual preview. The card demonstrates account-based daily progress and a learning streak.

Prepared by	Arthi.S
Degree	B.Sc. Computer Science with Artificial Intelligence
College	S.A. College of Arts and Science

Live website: learnsphere-ai-ddv4.onrender.com

Contents

This guide is organized into exactly 15 numbered sections.

Purpose and scope

Author profile

Problem and project objectives

How learners use the platform

Feature modules

Website page: Home

Website page: Learner Dashboard

Website page: Aptitude Arena

Website page: Code Lab

Website page: Interview Hub

English, AI support, and Placement Plan

Languages and technology stack

Architecture and data model

Authentication, credentials, and progress

Deployment, limits, and next steps

SECTION 01 OF 15

Purpose and scope

LearnSphere AI is a web application for learning and placement preparation. It brings practice activities, coaching tools, and saved progress together in one student workspace.

This document explains the user experience, key website pages, code technologies, data flow, account security, progress tracking, and the Render deployment. The screenshots are labeled as page previews when they illustrate an interface rather than a live browser capture. The Render dashboard image is the service screenshot supplied for this project.

SECTION 02 OF 15

Author profile

Prepared by **Arthi.S**, a student pursuing **B.Sc. Computer Science with Artificial Intelligence** at **S.A. College of Arts and Science**.

This project demonstrates how a web application can combine core software engineering, data persistence, and generative AI services to support practical learning.

SECTION 03 OF 15

Problem and project objectives

Learners often need to switch between separate tools for aptitude questions, coding practice, interview preparation, and communication coaching. LearnSphere AI places those activities in one guided learning experience.

- Provide structured placement practice across several skill areas.
- Offer clear feedback and AI-assisted explanations where configured.
- Save practice results to a learner account.
- Show a daily progress indicator and a learning streak based on completed activities.
- Keep navigation simple so learners can move from a goal to a practice page quickly.

SECTION 04 OF 15

How learners use the platform

A visitor arrives at the home page, reviews the available learning areas, and registers or logs in. After login, the learner opens a practice module, completes an activity, and saves the result. Flask routes validate the request, the relevant feature logic runs, and SQLAlchemy stores the result against the signed-in account.

The dashboard summarizes recorded work. The homepage progress card gives a short view of today’s activity. Merely opening a learning page does not count as completed practice.

SECTION 05 OF 15

Feature modules

Aptitude Arena: short quizzes in quantitative aptitude, logical reasoning, Python, operating systems, DBMS, and computer networks.

Code Lab: coding prompts and a Python runner for trying solutions.

Interview Hub: written interview practice, answer review signals, and a 100-level challenge track.

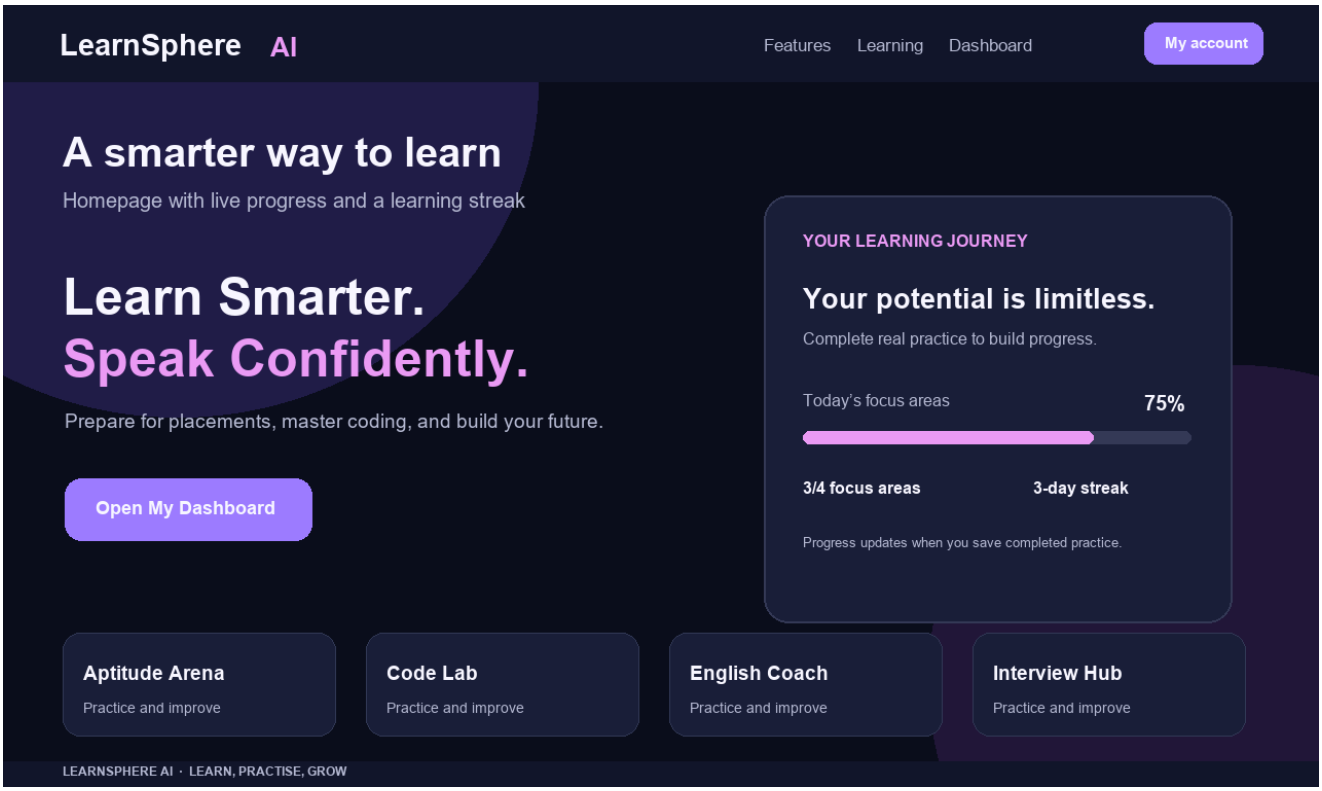
English Coach: writing and pronunciation feedback.

AI Coach and Study Assistant: career guidance and topic explanations.

Placement Plan: daily suggested tasks across practice categories.

SECTION 06 OF 15

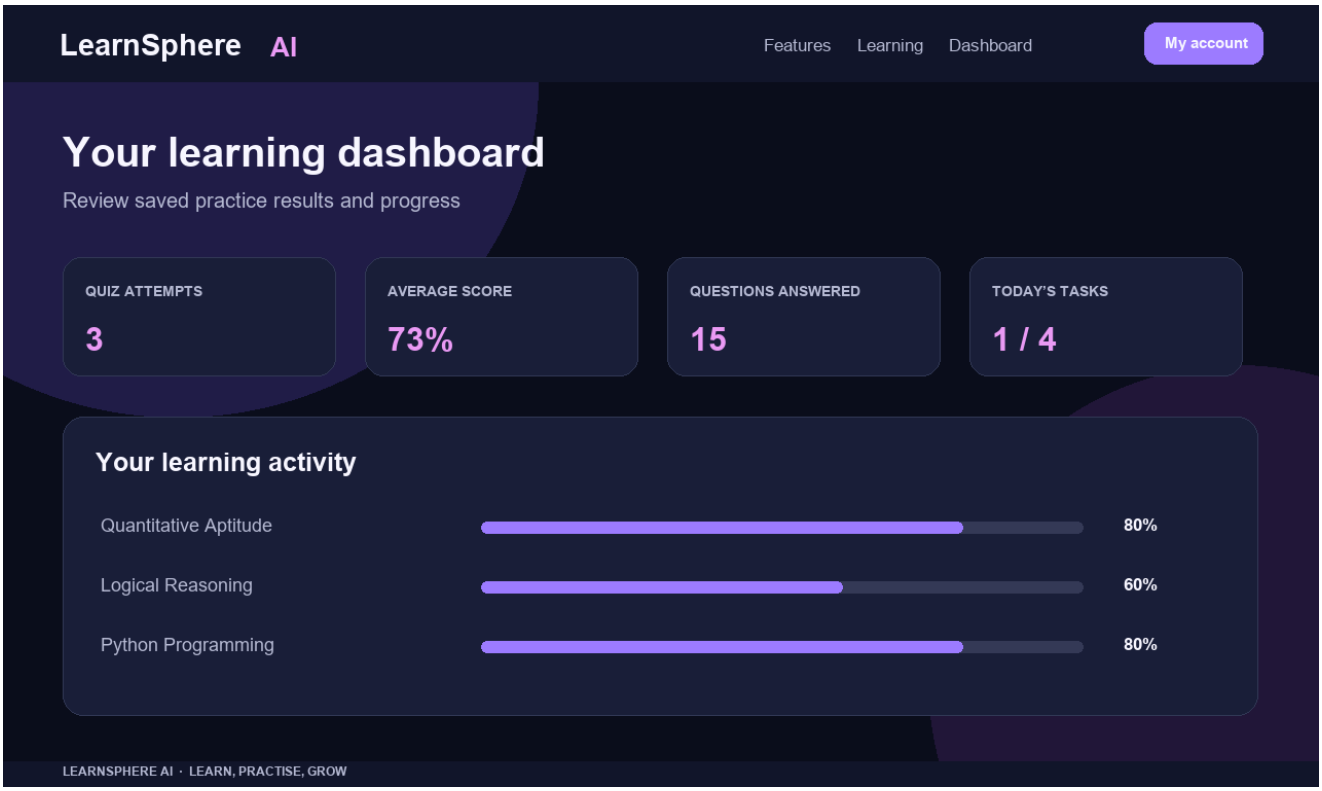
Website page: Home



The home page introduces LearnSphere AI and links to its learning areas. For signed-in learners, the journey card displays today's completed focus areas as a percentage and shows the current streak. Guests see a sign-in prompt so they do not mistake example values for personal progress.

The four progress categories are aptitude, coding, interview practice, and communication. Each category counts once for a learner on a calendar day, even if the learner completes several activities in it.

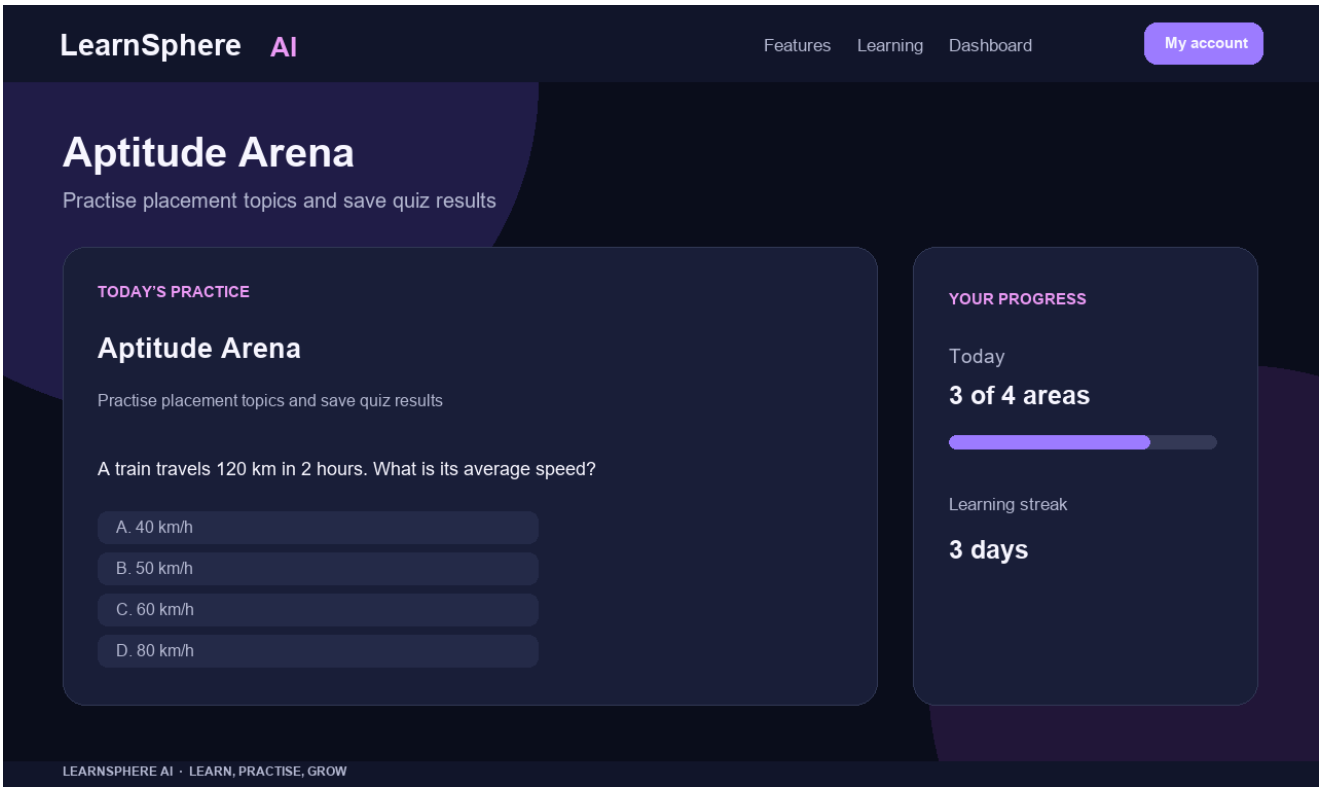
Website page: Learner Dashboard



The dashboard brings saved results and practice totals together. It can show quiz attempts, averages, answered questions, daily plan completion, and learning activity. It gives a learner a place to review what has been recorded for their account.

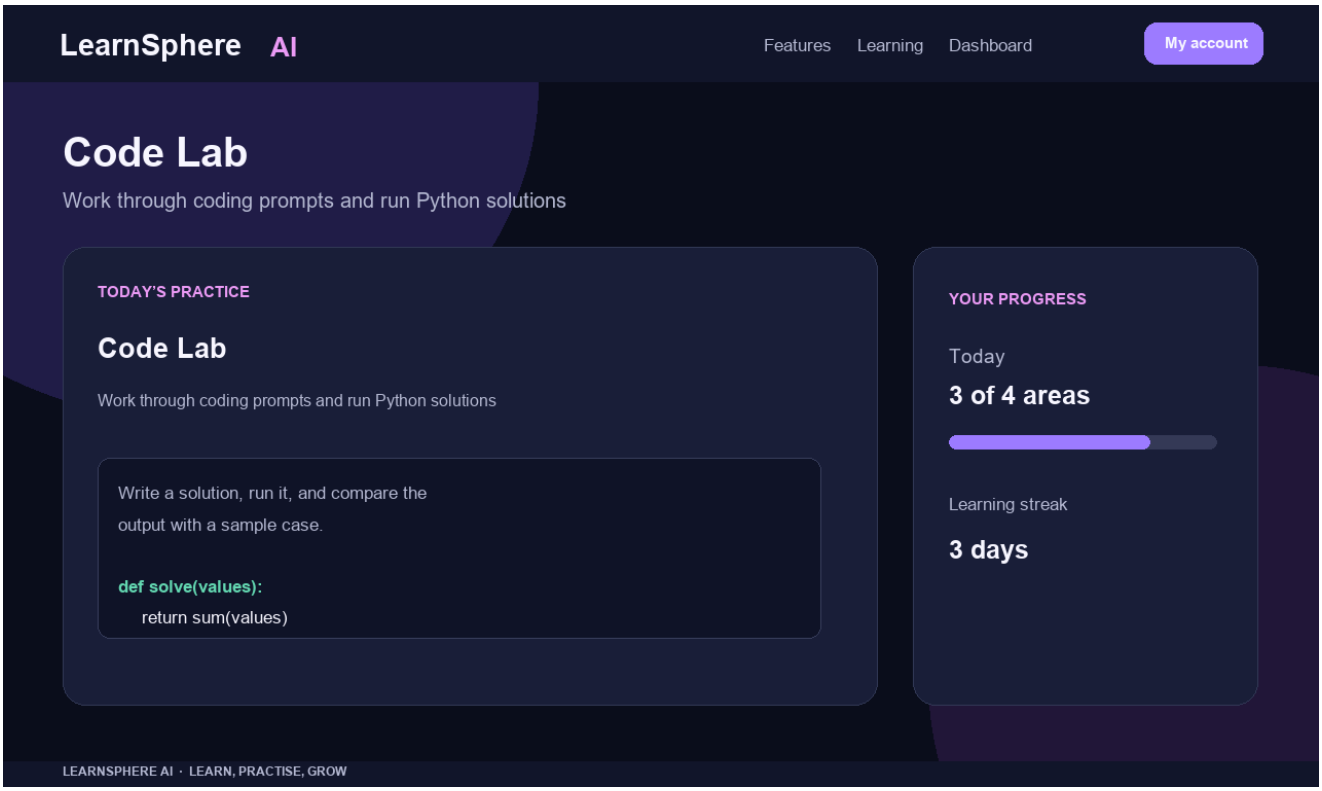
SECTION 08 OF 15

Website page: Aptitude Arena



The Aptitude Arena page lets a learner choose a topic and difficulty, then practise a five-question quiz. After the client submits a valid score, the server saves the QuizAttempt and records aptitude activity for today. A saved score can contribute to the dashboard topic averages.

Website page: Code Lab

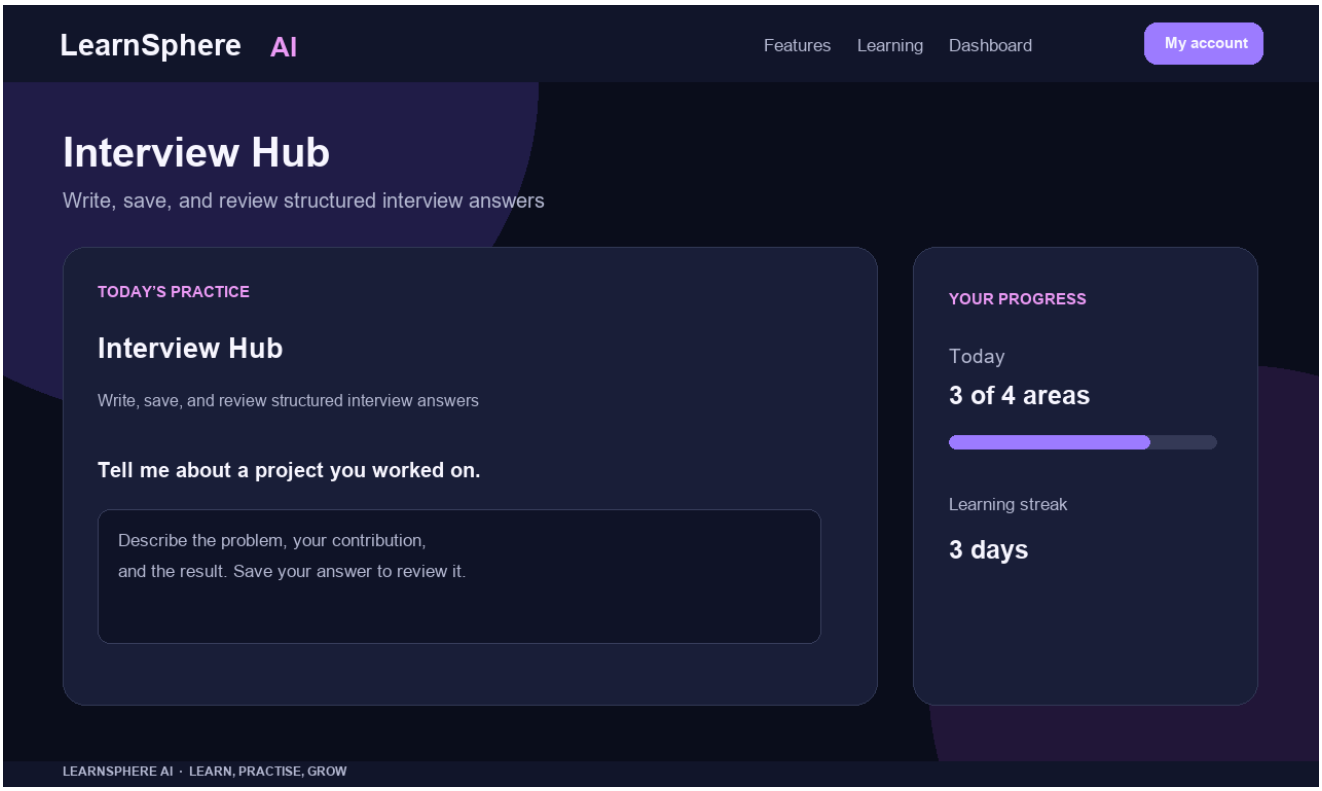


Code Lab presents a challenge and a Python editor/runner flow. The runner writes submitted code to a temporary file and invokes a separate Python process with a short time limit and size limits. Completing a challenge records coding activity.

The runner is intended for learning demonstrations. A subprocess timeout is not a strong security sandbox for arbitrary public code. Production use with untrusted users needs stronger isolation and resource controls.

SECTION 10 OF 15

Website page: Interview Hub



Interview Hub offers prompts such as project deep dives, teamwork, and problem solving. A learner writes a response, saves it, and sees a lightweight review of signals such as a specific example, personal contribution, impact, and takeaway. Saving an answer counts toward interview progress.

SECTION 11 OF 15

English, AI support, and Placement Plan

English Coach accepts a written or spoken-practice input and requests language feedback from the configured AI service. Successful feedback records communication practice.

AI Coach helps with career preparation. **Study Assistant** answers learning questions and can explain topics. Successful assistant exchanges are saved to the account and advance a learning streak, but do not add to one of the four named homepage focus areas.

Placement Plan proposes tasks across aptitude, coding, interview, and communication. When a learner marks a task complete, the completion is stored for that day and contributes to its matching focus area.

AI features require a valid Gemini API key configured on the server. If it is missing or the provider is unavailable, AI-backed actions cannot return their normal feedback.

SECTION 12 OF 15

Languages and technology stack

Technology	How the project uses it
Python	Flask routes, account logic, feature services, database models, and code challenge process management.
HTML + Jinja2	Server-rendered page structure and dynamic values such as progress, streak, menus, and saved records.
CSS	Layout, responsive page styling, colors, cards, buttons, and progress visuals.
JavaScript	Browser-side quiz and interactive feature behavior, including asynchronous result submission.
SQL databases	Relational persistence through SQLAlchemy. Local setup uses SQLite; the Render Blueprint configures PostgreSQL.

Main Python dependencies include Flask, Flask-Login, Flask-SQLAlchemy, python-dotenv, google-genai, httpx, Gunicorn, and psycopg. Jinja2 is the template engine used by Flask.

SECTION 13 OF 15

Architecture and data model

Request flow: browser request → Flask route → validation and authentication → feature service and/or SQLAlchemy model → rendered HTML or JSON response.

User stores the account identity, password hash, role, and streak fields. **QuizAttempt** stores quiz topic and score. **InterviewPracticeSession** stores written practice answers. **InterviewChallengeCompletion** stores completed challenge levels. **PlacementPlanCompletion** stores plan task dates. **StudyAssistantMessage** stores saved AI conversation messages. **LearningActivity** stores a user, category, and date for daily homepage progress.

The app creates SQLAlchemy tables at startup. The additional LearningActivity table lets the homepage query the current day without treating page visits as achievements.

SECTION 14 OF 15

Authentication, credentials, and progress

Flask-Login maintains the signed-in user session and protects learner routes. Registration stores a Werkzeug password hash; login checks the entered password against the hash. The plaintext password is not stored.

Password reset links use a signed, time-limited token created with itsdangerous. The reset route validates the token before allowing the password to be changed. Flask SECRET_KEY protects session and token signing.

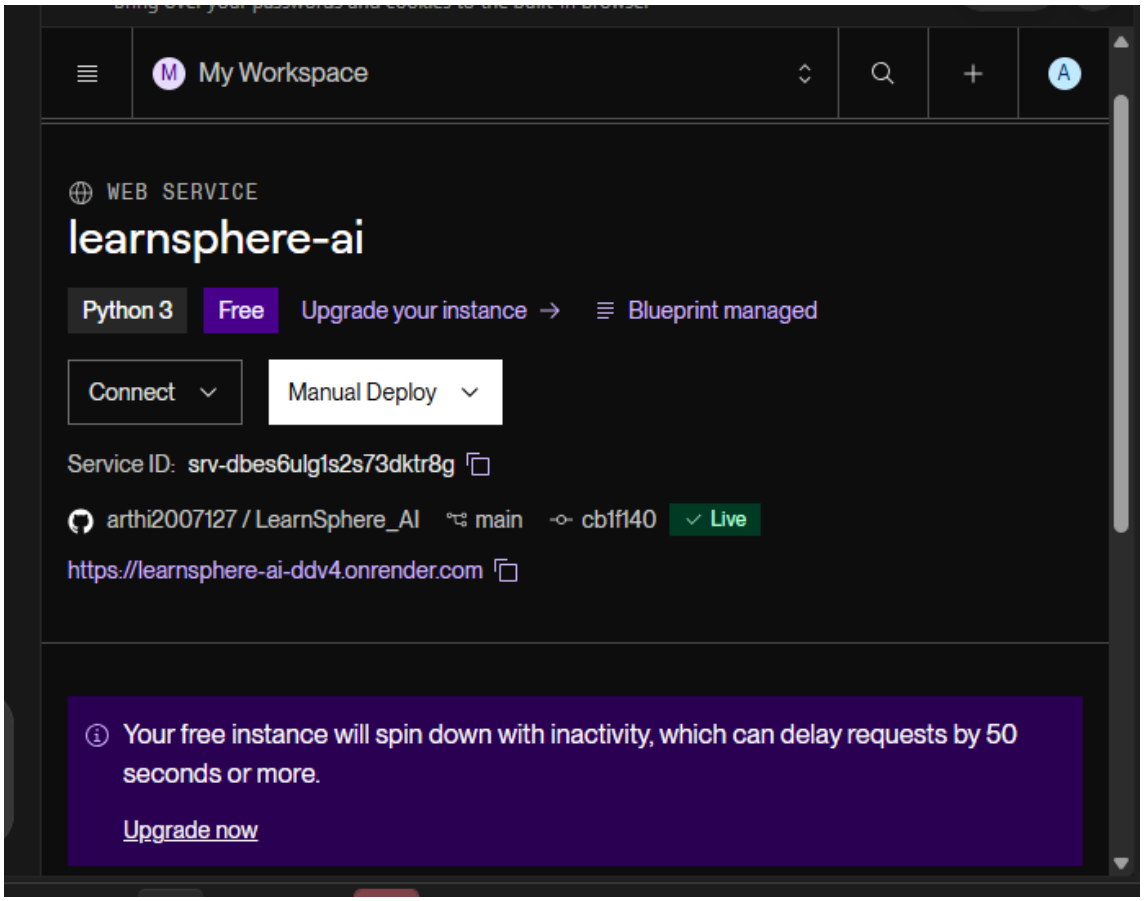
Gemini API credentials and database connection strings are server-side configuration values. In Render, set GEMINI_API_KEY and SECRET_KEY in the service environment; do not embed them in templates or browser JavaScript, or commit them in .env.

Streak behavior: the first completed action on a date advances the streak once. If the previous practice date was yesterday, the streak increases by one; otherwise it starts at one. Additional activity on the same date does not increase it again.

SECTION 15 OF 15

Deployment, limits, and next steps

The app is hosted at https://learnsphere-ai-ddv4.onrender.com. The Render service is connected to the LearnSphere_AI GitHub repository. The supplied dashboard capture shows the service marked Live and the public URL.



Render service dashboard capture supplied with the project.

- The free web instance can sleep after inactivity, so a first request may be delayed.
- The free PostgreSQL database has a limited lifetime; migrate or back up before relying on it for long-term records.
- Set production secrets in Render environment settings and keep them out of Git.
- Next improvements could include stronger code execution isolation, clearer account settings, automated backups, and broader progress history.

This guide contains exactly 15 numbered project sections. The page visuals are labeled previews; only the Render service image is a screenshot supplied with the project.